PLC PROJECT
DENSE LAYER

Shams Tarek
Rayan Abdelhady
Ahmed Tarek
Mohamed Ezzat
Aya El Bakri

MINST training

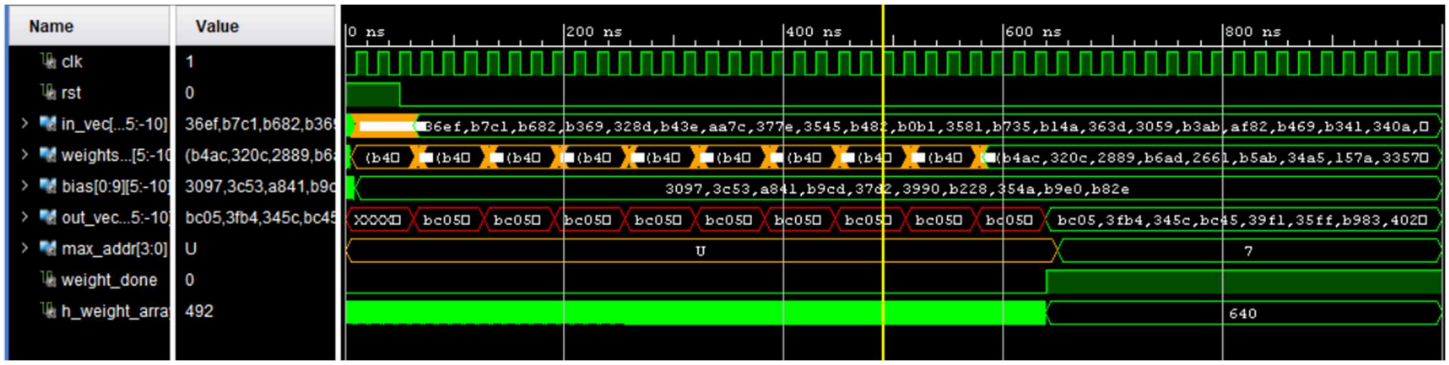
- **Python code**

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Model

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test  = x_test.reshape(-1, 28, 28, 1) / 255.0

model = models.Sequential([
   layers.Conv2D(2, (5,5), activation='relu', input_shape=(28,28,1)),
   layers.MaxPooling2D((2,2)),
   layers.Conv2D(4, (5,5), activation='relu'),
   layers.MaxPooling2D((2,2)),
   layers.Flatten(input_shape=(28,28,1)),
   layers.Dense(10, activation='sigmoid')
])

model.compile(optimizer='adam',
          loss='sparse_categorical_crossentropy',
          metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=30, validation_data=(x_test, y_test))

test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
```

- **Results**

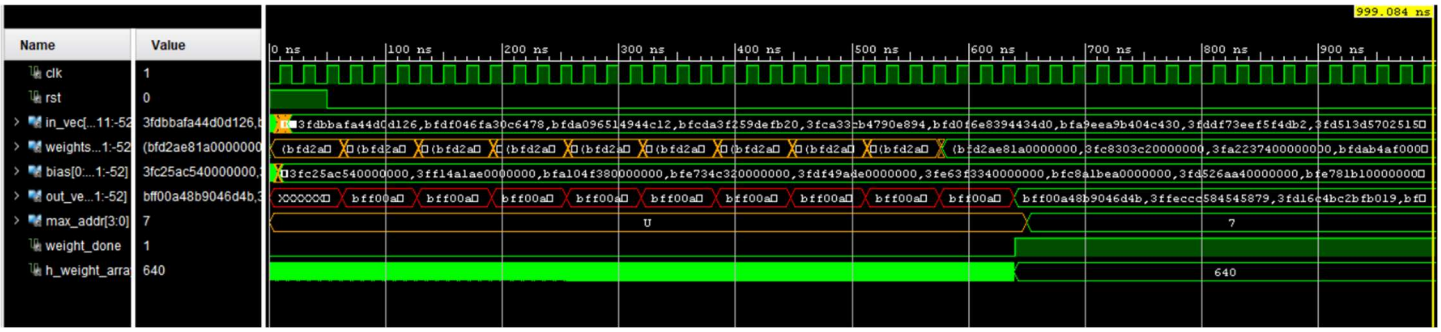
```
Test accuracy: 0.9747999906539917
```

DENSE LAYER FLOATING POINT 16

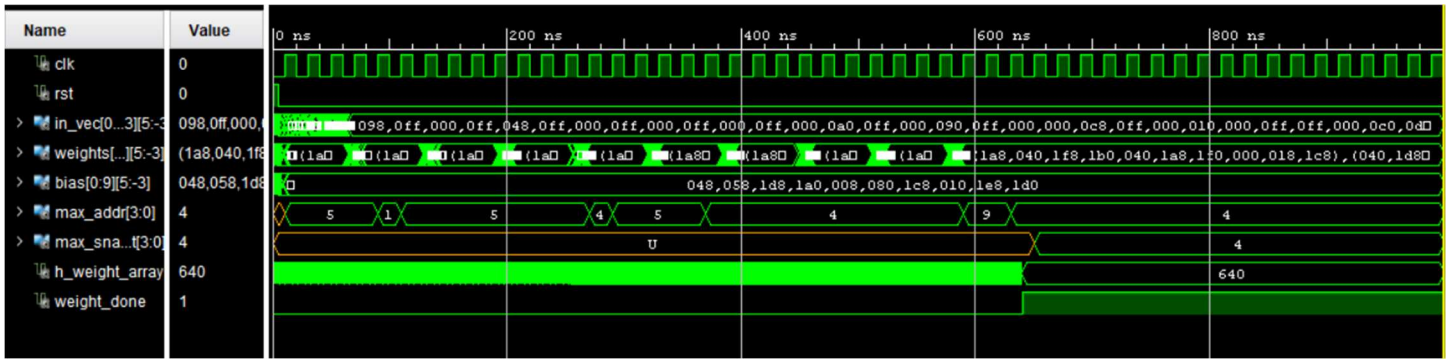


DENSE LAYER FLOATING POINT 32

DENSE LAYER FLOATING POINT 64



DENSE LAYER FIXED 8



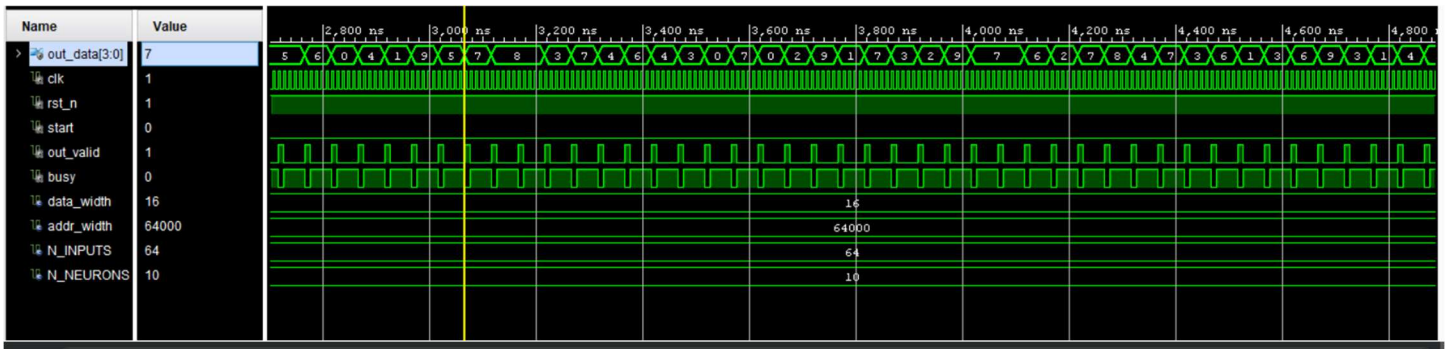
DENSE LAYER FIXED16

	Floating point 16	Floating point 32	Floating point 64	Fixed 8	Fixed 16
Speed	8.321 MHZ	6.339 MHZ	4.34 MHz	295 MHZ	282 MHz
Dynamic Power	12.358 W	26.779 W	82.178 W	1.086 W	2.807 W
Area	LUT: 725624 IO: 10340 MUXES: 523	LUT: 1424178 IO: 17460 MUXES: 1070	LUT: 4449268 IO: 38486 MUXES: 8700	LUT: 70009 IO: 6521	LUT: 161150 IO: 12143

As floating point precision increases from FP16 to FP32 to FP64, the designs become slower, larger, and more power is consumed, while fixed-point implementations remain much more hardware efficient. This happens because floating point arithmetic is inherently complex it needs extra logic for comparing exponents, aligning mantissas, normalizing results, rounding, and handling special cases. All of this adds depth to the logic, creates more routing congestion, and increases switching activity. As a result, higher precision floating-point circuits grow substantially, leading to lower operating speeds and higher resource and power usage for instance, the speed drops from 8.321 MHz for FP16 to 4.34 MHz for FP64, and power consumption rises from 12.358 W to 82.178 W.

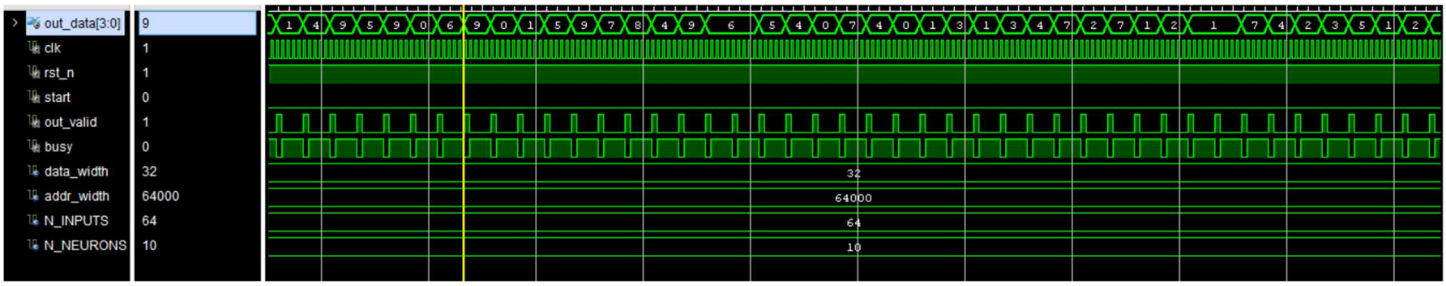
On the other hand, fixed point arithmetic is much simpler, behaving like straightforward integer operations. Its data paths are shorter and more regular, with no exponent handling, which keeps the area small and performance generally high. For the fixed point designs in particular, both versions share the same architecture and only differ in bit width. So, the changes in speed, area, and power mainly come from wider data paths due to increasing the word length enlarging the logic, routing, and switching capacitance, which in turn increases resource use and dynamic power and impacts achievable speed.

FLOATING POINT 16 FSM



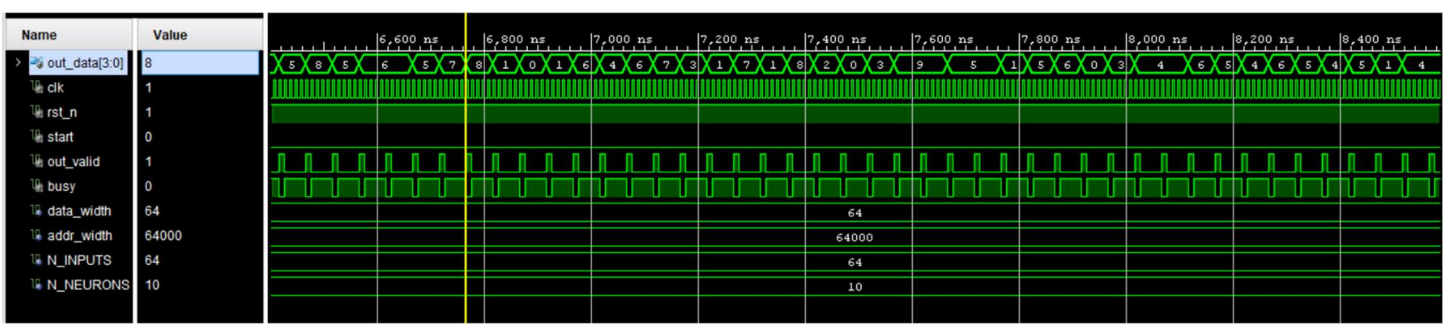

```
[4]
✓ 0s
accuracy = compute_accuracy("output_file_fp16.txt", "colab_predictions.txt")

Total samples : 10000
Correct       : 9999
Accuracy      : 99.99%
```

FLOATING POINT 32 FSM



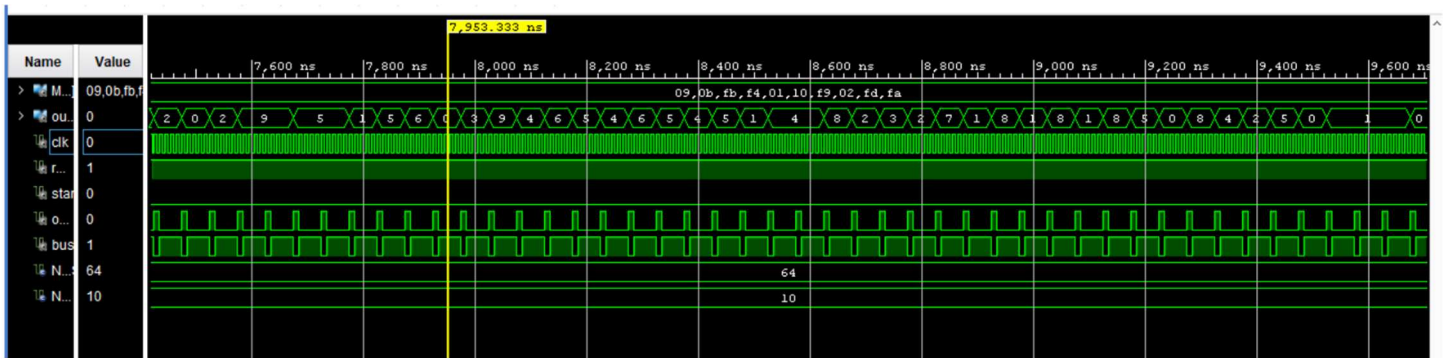

```
[8]
✓ 0s
accuracy = compute_accuracy("output_file_fp32.txt", "colab_predictions.txt")

Total samples : 10000
Correct       : 10000
Accuracy      : 100.00%
```

FLOATING POINT 64 FSM



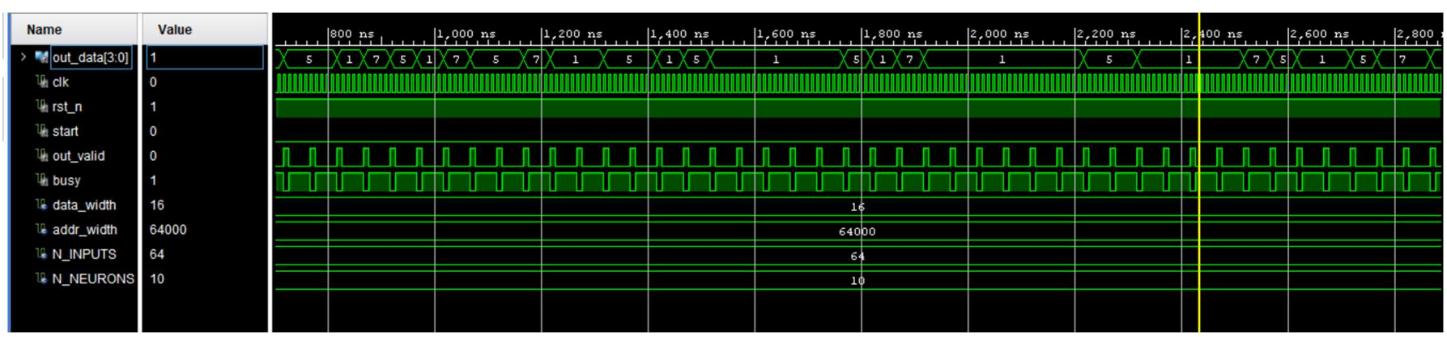
```
[10]
accuracy = compute_accuracy("output_file_fp64.txt", "colab_predictions.txt")

Total samples : 10000
Correct       : 10000
Accuracy      : 100.00%
```

FIXED 8 FSM




```
[6]
accuracy = compute_accuracy("output_file_fx8_10k_new_new.txt", "colab_predictions.txt")

Total samples : 10000
Correct       : 8291
Accuracy      : 82.91%
```

FIXED 16 FSM




```
[25]
accuracy = compute_accuracy("output_file_fx16_10k_new.txt", "colab_predictions.txt")

Total samples : 10000
Correct       : 9591
Accuracy      : 95.91%
```

	Floating point 16	Floating point 32	Floating point 64	Fixed 8	Fixed 16
Python accuracy	97.4%				
VHDL accuracy	97.39026 %	97.4 %	97.4 %	80.75434 %	93.41634 %

These results compare the accuracy of a neural network implemented in Python with various numerical representations in VHDL. The Python implementation achieves an accuracy of 97.4%, serving as the reference. Using floating-point representations in VHDL (16-bit, 32-bit, and 64-bit) closely matches this reference, with accuracies ranging from 97.39026% to 97.4%, indicating that floating-point formats preserve the network's precision effectively. In contrast, fixed-point representations show a noticeable drop in accuracy: Fixed 8 achieves approximately 80.75% and Fixed 16 reaches 93.42%. This drop occurs because fixed-point formats have limited precision and a restricted dynamic range. In low-bit fixed-point numbers, many values are rounded or truncated, introducing quantization errors in both weights and activations. These errors accumulate through the network layers, causing significant deviations from the intended computations. Additionally, small differences in input scaling or intermediate sums can lead to incorrect neuron outputs, which further reduces overall network accuracy. Essentially, Fixed 8 and Fixed 16 cannot represent the fine weight and activation values accurately enough, whereas floating-point formats can, which is why their accuracy remains nearly identical to the Python baseline. Overall, the results highlight that floating-point formats are reliable for preserving accuracy in hardware implementations, while low-bit fixed-point formats require careful design to avoid severe degradation.